ViMax 从入门到精通

> **版本与核实信息**
>
> - 教程基于版本：ViMax v1.2.0（2026 年 7 月 20 日发布，Web Workspace）
> - 核实日期：2026 年 10 月 21 日
> - 信息来源：GitHub 仓库 HKUDS/ViMax（readme.md、README_ZH.md、pyproject.toml、vimax 启动脚本、prompts/workflow.md、v1.2.0 Release 说明）、论文《ViMax: Agentic Video Generation》（arXiv: 2606.07649）；部分实践经验参考第三方上手指南，文中已标注
> - 适用环境：Linux、Windows；Python 3.12 及以上（由 pyproject.toml 声明）；Web UI 与 TUI 需 Node.js 18 及以上
> - 项目许可证：MIT
> - 说明：ViMax 迭代较快，涉及密钥、费用与最新功能的内容请以官方文档与仓库为准。文中标注「示意」的输出为编写时的合理示例，非逐字实录。

ViMax 是香港大学 HKUDS 团队开源的智能体视频创作框架。本教程面向希望从零开始掌握 ViMax 的创作者与开发者，假设读者具备基本的命令行使用经验，不要求人工智能或视频制作背景。全书按「入门篇、进阶篇、高级篇」三部分递进：入门篇完成安装配置并跑通第一条视频；进阶篇掌握三种入口与多条工作流；高级篇解决一致性、成本与故障排查问题，最终完成一个综合实战项目。

第一部分 入门篇

第 1 章 认识 ViMax：定位、能力边界与整体架构

许多使用者第一次接触 ViMax 时，都会把它误认为一个「文生视频模型」。在投入安装与费用之前，先厘清它是什么、不是什么，可以避免后续的预期偏差与无用支出。本章介绍 ViMax 的定位、它解决的核心问题、四大创作能力与整体架构，为全书奠定概念基础。

1.1 ViMax 是什么

ViMax 是一个**智能体驱动的视频创作框架**（agentic video generation framework）。它本身不包含任何图像或视频生成模型，也不在本地渲染像素；它把视频创作拆解为一串智能体步骤——扩写故事、抽取角色、生成分镜、挑选参考图、逐镜头出片、最终拼接——每一步都调用使用者配置好的外部模型 API。

因此，ViMax 的画质上限取决于所接入模型的画质上限；ViMax 自己负责的，是中间那段最繁琐的编排工作：让角色跨镜头不换脸、让场景前后衔接、把一句话的创意变成结构化的多镜头叙事。

一个容易混淆的要点：**ViMax 免费开源，但运行它几乎每一步都在调用付费的模型服务。**安装本身不花钱，生成过程会按调用次数向模型供应商付费。成本问题在第 11 章专门讨论。

1.2 它解决什么问题

官方 README 归纳了当前 AI 视频生成的三类局限，ViMax 的设计针对这三点展开：

现有方式的局限	具体表现	ViMax 的应对
只能生成短片段	主流模型单次只能生成数秒画面	多场景、多镜头拆解后逐段生成，再拼接成片
一致性混乱	角色与场景在跨镜头时外观不可控	角色参考图、首帧驱动与依赖感知的一致性追踪
只关注画面	缺少剧本、音频与叙事结构	从叙事输入出发，先规划故事、剧本与分镜，再生成画面

论文摘要进一步指出，ViMax 采用分层叙事引擎（配合检索增强生成，Retrieval-Augmented Generation，简称 RAG，即让模型在生成时检索相关背景材料以提高全局连贯性的技术）维持故事全局连贯，并以依赖感知的视觉一致性机制跨场景追踪角色与环境状态，由视觉语言模型（Vision-Language Model，简称 VLM，指同时理解图像与文本的模型）持续监测并修正叙事与画面质量。

1.3 四大创作能力

ViMax 当前提供四条能力线，前三条已有完整工作流：

能力	输入	输出
Idea2Video	一段简短创意描述	结构化故事、角色、剧本、分镜、镜头设计与成片
Script2Video	明确的剧本文本	忠实于剧本意图的多场景、多镜头视频
Novel2Video	长篇小说文本	经叙事压缩与角色追踪的分集视觉内容
AutoCameo	参考照片（人物或宠物）	将该形象融入生成故事并保持外观一致

需要特别说明 AutoCameo 的实现状态：截至本教程核实日期（2026 年 10 月），有第三方资料对 main 分支的检查显示，仓库中尚未找到 AutoCameo 的独立入口脚本与对应管线实现；该能力在早期版本中曾被标注为「Coming Soon」。使用者如需此功能，请在仓库中确认最新实现状态后再投入配置，不要默认其开箱可用。

1.4 三种使用入口

自 v1.2.0 起，ViMax 提供三种入口，共用同一套底层管线，差别只在交互方式：

入口	适合场景	交互方式
Agent TUI	日常主力创作，边聊边改	终端对话，可保存与恢复会话
脚本直跑	理解管线原理、固定任务复跑	编辑配置文件与脚本内变量后运行
Web UI	可视化管理、预览分镜与渲染进度	浏览器操作，与 TUI 共用同一运行时

三种入口在进阶篇分别用一章展开（第 5、6、7 章）。

1.5 整体架构与工作流

ViMax 的核心工作方式是**流水线**：一次创作被拆解为若干环节，前一环节的输出是后一环节的输入。这些环节及其依赖关系构成一张有向无环图（Directed Acyclic Graph，简称 DAG，即节点单向连接、不构成回路的图结构）。以 Idea2Video 为例，官方定义的流程如下：

```
input_idea（输入创意）
  -> project_brief（项目简报）
  -> characters（角色设计）
  -> script（剧本）
  -> storyboard（分镜）
  -> shot_decomposition（镜头拆解）
  -> camera_tree（机位树）
  -> frame_prompts（首帧提示词）
  -> keyframes（关键帧图像）
  -> video_clips（镜头视频）
  -> final_video（成片）
```

理解这张图对全书后续内容至关重要：排查故障时要按图定位中断环节（第 12 章），控制成本时要按图估算调用次数（第 11 章），断点续跑时要以图中节点对应的产物文件为依据（第 9 章）。

1.6 能力边界

使用前应明确以下边界，均为客观限制：

1. **不内置生成模型**：必须自备对话模型、图像生成模型与视频生成模型的 API 密钥；
2. **单镜头时长受底层模型限制**：主流视频模型单次生成数秒，长片依靠多镜头拼接达成；
3. **依赖付费 API**：规划阶段的对话模型调用、每个镜头的图像与视频生成均产生费用；
4. **AutoCameo 尚无公开的独立入口**（见 1.3 节说明）；
5. **生成结果具有随机性**：同一输入多次运行可能产出不同画面，迭代与筛选是常态。

本章小结：ViMax 是「编排层」而非「生成层」，价值在于把叙事规划与视觉一致性自动化；使用它需要自备三类模型服务并承担相应 API 费用；后续所有章节都围绕它的流水线（DAG）展开。

第 2 章 环境准备与安装

定位清楚之后，下一步是把环境装好。ViMax 用 uv 管理 Python 环境，用 Node.js 驱动 TUI 与 Web UI 的前端，本章完成全部依赖的安装与验证，并说明各目录的用途。完成本章后，读者应能在本机成功执行 `uv sync` 并理解仓库根目录下各子目录的作用。

2.1 系统要求

官方声明的支持系统为 Linux 与 Windows。各组件要求如下：

组件	要求	说明
操作系统	Linux 或 Windows	macOS 未列入官方支持列表
Python	3.12 及以上	由 pyproject.toml 声明 `requires-python = ">=3.12"`，uv 会自动准备匹配版本
uv	最新版即可	Python 包与环境管理工具，见 2.2 节
Git	任意近期版本	用于克隆仓库
Node.js	18 及以上	仅 TUI 与 Web UI 需要；纯脚本直跑模式不需要
磁盘空间	建议 5 GB 以上	依赖（含 PyTorch CUDA 版本）与中间产物占用较大

关于硬件：生成环节全部由云端 API 完成，本地不需要 GPU；但 pyproject.toml 将 PyTorch 指向 CUDA 12.8 索引（Linux 与 Windows 平台），依赖下载体积较大，预留磁盘空间即可。

2.2 安装 uv

uv 是 Astral 开发的 Python 包与虚拟环境管理工具，特点是速度快且能按项目声明自动创建虚拟环境、自动安装匹配版本的 Python。ViMax 官方指定 uv 作为环境管理方案，安装方法以官方文档（https://docs.astral.sh/uv/getting-started/installation/ ）为准，常用命令如下。

Windows（PowerShell）：

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Linux：

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

安装完成后重开终端，验证版本：

```
uv --version
```

预期输出（示意）：

```
uv 0.8.x
```

2.3 克隆仓库并同步依赖

在选定的工作目录执行：

```
git clone https://github.com/HKUDS/ViMax.git
cd ViMax
uv sync
```

`uv sync` 会完成三件事：按 pyproject.toml 声明创建名为 `.venv` 的虚拟环境；下载并安装匹配的 Python 3.12 解释器（如本机没有）；安装全部依赖，包括 langchain（智能体编排框架）、moviepy（视频拼接库）、opencv-python（图像处理库）、faiss-cpu（向量检索库）、google-genai、openai 等。首次执行下载量大，需要数分钟。

验证 Python 环境可用：

```
uv run python --version
```

预期输出（示意）：

```
Python 3.12.x
```

后续凡在仓库根目录运行 Python，都推荐使用 `uv run python …` 形式，uv 会自动使用 `.venv` 环境；也可以先激活虚拟环境再用系统 `python` 命令（Linux：`source .venv/bin/activate`；Windows PowerShell：`.venv\Scripts\Activate.ps1`）。

2.4 前端依赖（使用 TUI 或 Web UI 时）

TUI 与 Web UI 是两个独立的 Node.js 前端，需要分别安装依赖：

```
# TUI 前端（TUI 必需）
cd ui
npm install
cd ..

# Web UI 前端（使用 Web UI 时安装）
cd web
npm install
cd ..
```

这一步不可省略：ViMax 的启动脚本 `vimax` 在启动 TUI 前会检查 `ui/node_modules/.bin/tsx` 是否存在，缺失时直接报错终止；Web 模式同理检查 `web/node_modules/.bin/vite`。

2.5 仓库目录速览

与使用者直接相关的目录与文件如下：

路径	用途
`configs/`	配置文件目录，含 `agent.example.yaml`（TUI/Web 共用示例配置）与 `idea2video.yaml`、`script2video.yaml`（脚本直跑配置）
`main_idea2video.py`	Idea2Video 脚本直跑入口
`main_script2video.py`	Script2Video 脚本直跑入口
`vimax`	统一启动脚本（bash），提供 `vimax tui` 与 `vimax web` 两组子命令
`ui/`	TUI 前端源码（基于 React 的终端渲染框架 Ink）
`web/`	Web UI 前端源码
`prompts/workflow.md`	工作流的权威说明文档，定义各工作流的 DAG 与产物规范
`agent_runtime/`	Agent 运行时代码，将工作流包装为对话工具
`.working_dir/`	运行时生成的产物目录（首次运行后出现）
`.vimax/`	运行时生成的会话索引与偏好记录（首次运行后出现）

本章小结：安装的核心是 `uv sync` 一条命令；若要使用 TUI 或 Web UI，还需分别执行 `ui` 与 `web` 目录下的 `npm install`；所有日常命令都在仓库根目录执行。

第 3 章 配置三类模型服务

ViMax 自己不生成任何像素，规划、配图、出片全部依赖外部模型服务，因此配置三类模型服务是跑通前最关键、也最容易出错的一步。本章逐字段讲解配置文件的结构，说明密钥的安全存放方式，并给出第一轮选型建议。完成本章后，读者应备好一份可运行、密钥不落盘或已受保护的配置。

3.1 三类模型的分工

类别	在流水线中的职责	对质量的影响
对话模型（LLM）	故事扩写、剧本与分镜生成、镜头规划、图像挑选裁判	决定叙事质量与规划合理性
图像生成模型	角色参考图（三视图）、场景关键帧（首帧）	决定画面风格与角色一致性基础
视频生成模型	依据关键帧逐镜头生成视频片段	决定动态质量与成片质感

此外，v1.2.0 的 Web UI 供应商设置中还包含 embedding（文本向量化）与 reranker（重排序）服务的配置项，用于检索相关能力，本教程不深入展开，保持默认或按界面提示填写即可。

3.2 TUI 与 Web UI 的配置：agent.local.yaml

TUI 与 Web UI 共用一份私有配置。先从受 Git 跟踪的示例复制出本地配置（仓库只跟踪 `agent.example.yaml`，`agent.local.yaml` 被忽略，不会被提交）：

```
cp configs/agent.example.yaml configs/agent.local.yaml
```

Windows PowerShell 下 `cp` 是 `Copy-Item` 的别名，可直接使用。

随后编辑 `configs/agent.local.yaml`，填入三段配置，每段包含四个字段：

```yaml
llm:
  model_provider: openai
  model: <YOUR_LLM_MODEL>
  base_url: <YOUR_LLM_BASE_URL>
  api_key: <YOUR_API_KEY>

image:
  model: <YOUR_IMAGE_MODEL>
  base_url: <YOUR_IMAGE_BASE_URL>
  api_key: <YOUR_API_KEY>

video:
  model: <YOUR_VIDEO_MODEL>
  base_url: <YOUR_VIDEO_BASE_URL>
  api_key: <YOUR_API_KEY>
```

字段含义逐个说明：

字段	含义
`model_provider`	供应商协议类型，`openai` 表示使用 OpenAI 兼容接口协议
`model`	模型标识，如 `google/gemini-2.5-flash-lite-preview-09-2025`
`base_url`	API 服务地址，如 OpenRouter 的 `https://openrouter.ai/api/v1`
`api_key`	对应服务的密钥，从供应商控制台获取

OpenRouter 是一个模型聚合网关，用一份密钥即可调用多家供应商的模型，官方示例即以 OpenRouter 作为对话模型的 `base_url`。图像与视频服务也可以走类似聚合网关，或直连供应商官方 API，取决于所选服务的接口形式。

若不希望密钥写进文件，可以保持 `agent.local.yaml` 为空，改用环境变量提供密钥：

环境变量	对应配置
`VIMAX_LLM_API_KEY`	`llm.api_key`
`VIMAX_IMAGE_API_KEY`	`image.api_key`
`VIMAX_VIDEO_API_KEY`	`video.api_key`

密钥安全提醒：API 密钥等同于付费账户的凭据，泄露会导致他人消耗账户余额。应遵循三点：密钥只写入被 Git 忽略的 `agent.local.yaml` 或环境变量，绝不写入仓库跟踪的文件；不要把含密钥的截图或日志发布到公开场合；在供应商控制台为密钥设置消费限额。

3.3 脚本直跑模式的配置：idea2video.yaml 与 script2video.yaml

脚本直跑模式使用另一套配置文件，字段名与 Agent 模式不同。以 `configs/idea2video.yaml` 为例：

```yaml
chat_model:
  init_args:
    model: google/gemini-2.5-flash-lite-preview-09-2025
    model_provider: openai
    api_key: <YOUR_API_KEY>
    base_url: https://openrouter.ai/api/v1

image_generator:
  class_path: tools.ImageGeneratorNanobananaGoogleAPI
  init_args:
    api_key: <YOUR_API_KEY>

video_generator:
  class_path: tools.VideoGeneratorVeoGoogleAPI
  init_args:
    api_key: <YOUR_API_KEY>

working_dir: .working_dir/idea2video
```

两点重要差异需要解释：

1. **图像与视频模型以 `class_path` 指定**：这里填的是适配器类的完整路径。适配器（adapter）是 ViMax 中对接某个具体生成服务的封装类，同类服务可更换适配器而不改动其余流程。官方示例中，`tools.ImageGeneratorNanobananaGoogleAPI` 对接 Google 的图像生成服务，`tools.VideoGeneratorVeoGoogleAPI` 对接 Veo 视频生成服务；更换其他受支持的服务时，只需修改这一行及对应 `init_args`。
2. `working_dir` **指定产物落盘目录**：该工作流全部中间产物与成片都写入此目录，见第 9 章。

`configs/script2video.yaml` 结构相同，`working_dir` 指向脚本工作流的目录。

3.4 第一轮选型建议

以下为建议而非事实，理由附后：

1. **对话模型选低成本档位即可。**规划阶段调用次数多、单次任务难度中等，使用 Flash 级别的轻量模型（官方示例即为 gemini-2.5-flash-lite 一类）可以显著压低规划成本，叙事质量的损失相对有限。
2. **图像与视频模型在预算内选最好的。**这两类模型直接决定成片观感，且单镜头调用次数固定（每镜头至少一张关键帧、一段视频），费用可预估，值得把预算集中在此。
3. **首次运行保持最小规模。**用默认的小规模约束（见第 4 章）跑通全流程、验证配置正确性之后，再更换更强的模型与更大的规模，避免在配置错误的状态下消耗大额费用。

本章小结：三类模型各司其职；Agent 模式用 `agent.local.yaml`（llm、image、video 三段），脚本模式用各自的 yaml（class_path 指定适配器）；密钥优先用环境变量或被 Git 忽略的本地配置承载。

第 4 章 第一次成片：跑通官方示例

配置就绪后，最有价值的下一步是用最小成本完整跑通一次流水线：从一段创意出发，拿到成片文件，并理解过程中落盘的每一类产物。本章以透明度最高的脚本直跑方式完成首次运行。完成本章后，读者应能独立执行一次 Idea2Video 全流程，并说出每个产物文件对应 DAG 中的哪个环节。

4.1 运行前的检查清单

首次运行前逐项确认：

1. `uv sync` 已完成，`uv run python --version` 输出 3.12.x；
2. `configs/idea2video.yaml` 三段密钥已填写（或环境变量已设置）；
3. 明确本次为小规模试跑：保留「不超过 3 个场景」级别的约束，控制费用。

4.2 填写创意

打开仓库根目录的 `main_idea2video.py`，找到以下三个变量并修改（官方示例如下）：

```
idea = \
"""
If a cat and a dog are best friends, what would happen when they meet a new cat?
"""
user_requirement = \
"""
For children, do not exceed 3 scenes.
"""
style = "Cartoon"
```

三个变量的含义与写法要点：

变量	含义	写法要点
`idea`	创意描述，流水线的唯一种子输入	简短但要有具体冲突或转折，故事性由此扩展
`user_requirement`	创作要求，属于**硬约束**	「不超过 3 个场景」这类数字约束会被规划环节直接执行；首次运行建议保留小规模约束以控制费用
`style`	全局视觉风格	如 `Cartoon`（卡通），建议用英文常见风格词

注意：这三个变量写在文件里，不是命令行参数。执行 `python main_idea2video.py --idea "…"` 这类命令不会有任何效果，必须直接编辑文件。

4.3 运行

在仓库根目录执行：

```
uv run python main_idea2video.py
```

运行过程会依次执行 DAG 各环节，终端输出每个环节的进度。由于各环节间存在 API 调用，整个流程耗时从数分钟到数十分钟不等，取决于模型响应速度与镜头数量。中途可以中断（Ctrl-C），已完成环节的产物会保留，重跑同一命令将跳过这些环节（原理见第 9 章）。

4.4 理解输出产物

全部产物落在 `working_dir` 指定的目录（默认 `.working_dir/idea2video/`）下，按 DAG 顺序依次生成：

产物	对应环节	内容
`story.txt`	故事扩写	由创意扩写的完整故事文本
`characters.json`	角色设计	结构化角色清单，含是否在画面中出现的标记
角色三视图目录	角色参考	每个可见角色的正面、侧面、背面三张参考图及注册表
`script.json`	剧本	按场景组织的结构化剧本
逐场景子目录	分镜至镜头视频	每个场景下的分镜、镜头树、首末帧、镜头视频、转场
`final_video.mp4`	成片	由 moviepy 将全部镜头视频拼接的最终成片

其中「三视图」指同一角色在正面、侧面、背面三个角度下的参考图像，用于约束图像生成模型在不同镜头中保持角色外观一致，是 ViMax 一致性机制的基础（机制详解见第 10 章）。

4.5 观察跳过日志

若因网络或超时中断后重跑，终端会出现类似如下的日志（示意）：

```
story.txt exists, skipping.
characters.json exists, skipping.
```

含义是：该环节产物已存在，直接跳过，进入下一环节。这一机制保证中断重跑不会重复消耗已生成环节的费用。

本章小结：首次运行 = 填好三个变量后执行 `uv run python main_idea2video.py`；产物按 DAG 顺序落盘；`user_requirement` 是硬约束，首次运行用它把规模压到最小；中断重跑会自动跳过已完成环节。

第二部分 进阶篇

第 5 章 脚本直跑详解：Idea2Video 与 Script2Video

第 4 章跑通了 Idea2Video，但「脚本直跑」作为入口的全部能力还没有展开：它还包括把一份现成剧本搬上屏幕的 Script2Video，以及通过更换适配器接入不同生成服务的自由度。本章把这两个入口讲透，并说明四条能力线的实现状态。完成本章后，读者应能根据手里已有的素材（一句创意还是一份剧本）选择正确的工作流，并完成配置。

5.1 两条工作流的 DAG 对比

两条工作流共用从 `keyframes` 之后的渲染环节，差别在源头：

```
Idea2Video:
input_idea -> project_brief -> characters -> script -> storyboard
  -> shot_decomposition -> camera_tree -> frame_prompts -> keyframes
  -> video_clips -> final_video

Script2Video:
input_script -> characters -> storyboard
  -> shot_decomposition -> camera_tree -> frame_prompts -> keyframes
  -> video_clips -> final_video
```

Idea2Video 在角色之前多一步项目简报（project_brief）与故事扩写（script 前的故事文本），因为它要从一句话无中生有；Script2Video 的输入已是完整剧本，直接进入角色与分镜设计。二者不可混用：模糊的创意交给 Script2Video 会因缺少剧本文本而无法开始，官方规范明确要求不得从模糊创意虚构剧本文本。

5.2 main_script2video.py 的输入格式

编辑 `main_script2video.py` 的 `script`、`user_requirement`、`style` 三个变量。`script` 采用标准剧本格式——场景标题行（内景/外景、地点、时间）、动作描述、角色对白。官方示例（节选，完整示例见仓库文件）：

```
script = \
"""
EXT. SCHOOL GYM - DAY
A group of students are practicing basketball in the gym. John (18,
male, tall, athletic) is the star player, and he is practicing his
dribble and shot. Jane (17, female, short, athletic) is the assistant
coach.
John: (dribbling the ball) I'm going to score a basket!
Jane: (smiling) Good job, John!
#  (此处为官方示例的省略段，完整内容见仓库 main_script2video.py)
"""
user_requirement = \
"""
Fast-paced with no more than 20 shots.
"""
style = "Animate Style"
```

写法要点：

1. 场景标题遵循「EXT./INT.（外景/内景）+ 地点 + 时间」的 screenplay（剧本）惯例；
2. 角色首次出现时在动作描述中给出年龄、性别、外形等特征，供角色设计环节提取；
3. `user_requirement` 中的镜头数量约束（如「no more than 20 shots」）会被镜头拆解环节执行，是控制费用的第一道阀门。

运行方式与 Idea2Video 相同，配置文件换成 `configs/script2video.yaml`，产物落盘在其 `working_dir` 下，原始剧本会原样保存为 `script.txt`。

5.3 更换生成服务：适配器机制

脚本模式的图像与视频服务由 `class_path` 指定。截至核实日期，官方明确支持的生成服务包括：

服务	类型	适配器/接入方式
Google Veo	视频生成	`tools.VideoGeneratorVeoGoogleAPI`
Google 图像生成（Nanobanana）	图像生成	`tools.ImageGeneratorNanobananaGoogleAPI`
GPT Image 2（经 OpenRouter）	图像生成	2026 年 7 月 17 日更新加入
Seedance 2.0 Fast	视频生成	2026 年 7 月 17 日更新加入
MiniMax H3	视频生成	路线图条目，尚未标记完成

更换服务的操作是修改 `image_generator` 或 `video_generator` 的 `class_path` 与 `init_args`，其余流程不变。具体可用的类名以仓库 `tools/` 目录中的最新代码为准。

5.4 四条能力线的实现状态汇总

结合第 1 章的说明，截至 2026 年 10 月核实日期的状态如下：

能力	入口	状态
Idea2Video	`main_idea2video.py`、TUI、Web UI	可用
Script2Video	`main_script2video.py`、TUI、Web UI	可用
Novel2Video	TUI / Web UI（见第 8 章）	可用
AutoCameo	未发现公开入口脚本	宣传列入特性列表，实现待确认

本章小结：手里只有创意走 Idea2Video，已有剧本走 Script2Video；两条工作流的 DAG 差异在源头三步；换生成服务只改 `class_path`；AutoCameo 目前不应计入可用能力。

第 6 章 Agent TUI：交互式创作与会话管理

脚本直跑透明但僵硬：改一个想法就要改文件重跑。TUI（Terminal User Interface，终端用户界面）提供了对话式的创作方式——与一个 Agent（智能体，指能理解意图、规划步骤并调用工具完成任务的程序）讨论创意、逐轮修改规划、决定何时开始渲染，并随时保存恢复会

话。本章讲解 TUI 的工作方式、会话命令与它背后的运行时工具。完成本章后，读者应能在 TUI 中完成一次「讨论—规划—确认—渲染」的完整对话流程。

6.1 启动与会话命令

TUI 于 2026 年 6 月前后引入，v1.2.0 起与 Web UI 并列为标准入口。启动前确认第 2 章的 `ui` 目录依赖已安装，然后在仓库根目录执行：

```
./vimax tui
```

`vimax` 是仓库根目录下的 bash 启动脚本；Windows 平台需在 Git Bash 或 WSL 中执行。该脚本实际执行的命令是 `ui/node_modules/.bin/tsx ui/src/cli.tsx`（tsx 为 TypeScript 直接执行器），在 PowerShell 中也可直接调用等价路径。

会话（session）管理命令如下，同一命令的两种写法等价：

命令	作用
`./vimax tui`	恢复当前活跃会话并进入 TUI
`./vimax tui new`	新建并激活一个空会话
`./vimax tui resume`	恢复当前活跃会话
`./vimax tui resume <session_id>`	恢复指定的历史会话
`./vimax tui --new-session`	等价于 `tui new`
`./vimax tui --session <session_id>`	等价于 `tui resume <session_id>`

会话索引记录在 `.vimax/sessions.json`，用户偏好记录在 `.vimax/memory.md`（仅存偏好，不存业务数据）。

6.2 Agent Loop 与确认门槛

TUI 背后是 Agent Loop（智能体循环）：运行时把三条工作流包装成对话工具，Agent 依据使用者话语自行选择并调用。核心工具有两个：

工具	输入	用途
`vimax_narrative_planning`	`idea` 或 `script` 参数	创建或修订结构化文本产物（故事、角色、剧本、分镜等）
`vimax_novel_planning`	长篇小说文本	小说改编的分集规划（见第 8 章）
`vimax_render_video`	已存在的规划产物	执行从关键帧到成片的渲染环节
`view_image`	会话工作区内的图像	检查、对比或诊断已生成的画面

使用时必须了解**确认门槛（workflow confirmation gate）机制**：在调用任何规划工具之前，Agent 必须获得使用者对工作流的明确确认。表述模糊的想法、或只说「做一个短片」，都不会被当作确认；Agent 会先反问选择哪条工作流。这是防止走错工作流、浪费费用的保护机制。规范同时要求：模糊创意只能走 Idea2Video；提供明确剧本文本才可走 Script2Video。

另一个默认行为值得注意：对模糊创意，默认规划保持小规模——目标 1 个场景、3 至 5 个镜头，除非使用者明确要求更长、更多场景或更多镜头。这与脚本模式中 `user_requirement` 约束异曲同工，都是成本阀门。

6.3 一轮交互的典型过程

以下对话过程为示意，展示确认门槛与阶段划分：

```
使用者：我想做一个关于便利店夜班店员和一只流浪猫的故事。
Agent：请确认使用哪条工作流：idea2video、script2video 还是 novel2video?
使用者：用 idea2video。
Agent：（生成 project_brief、characters、script、storyboard 等文本产物，
       随后汇报规划完成，询问：继续修改，还是开始渲染？）
使用者：第二场镜头太少了，再加两个。
Agent：（修订对应产物，再次确认。）
使用者：开始渲染。
Agent：（调用 vimax_render_video，生成关键帧与镜头视频，输出成片。）
```

规划与渲染是两个阶段：文本产物齐备后，若使用者未要求生成，Agent 会停下询问，而不会擅自进入渲染。

6.4 产物目录规范

Agent 模式下，全部产物必须位于活跃会话目录之下：

```
.working_dir/<session_id>/
  idea2video/
  script2video/
  novel2video/
```

官方规范明确禁止在根级直接使用 `.working_dir/idea2video/` 这类路径读写（那是脚本直跑模式的领地，二者不可混用，否则会造成产物归属混乱）。

本章小结：TUI 的价值是把「改需求」变成对话；启动前必须装好 `ui` 依赖；工作流选择是显式确认门槛，模糊输入会被反问；文本规划与渲染分阶段推进；会话可保存、可恢复，产物一律归属于会话目录。

第 7 章 Web UI：可视化工作区

终端对话对部分使用者不够直观，v1.2.0 推出了浏览器工作区：命名项目、产物浏览、分镜预览、渲染进度与供应商设置集中在同一页面。本章介绍 Web UI 的启动、核心功能与远程访问方式。完成本章后，读者应能在浏览器中完成从建项目到预览成片的全流程，并掌握远程服务器场景下的端口转发配置。

7.1 启动

Web UI 与 TUI 共用同一套 Agent 运行时、会话、工具与 `configs/agent.local.yaml` 配置，因此无需重复配置。需要 Node.js 18 及以上，且 `web` 目录依赖已安装。在仓库根目录执行：

```
cd web
npm install    # 首次使用时
npm run dev
```

也可使用统一启动脚本（效果等同 `npm --prefix web run dev`）：

```
./vimax web
```

随后在浏览器打开 `http://127.0.0.1:4173`。

若 4173 端口被占用，用环境变量更换端口：

```
VIMAX_WEB_PORT=4174 npm run dev
```

7.2 核心功能

v1.2.0 Release 说明列出的能力如下：

功能	说明
命名项目	以项目为单位组织工作，Agent Loop 会话持久保存
产物浏览器	以结构化、非 JSON 原文的形式呈现文本产物
分镜预览	直接查看分镜画面的进展
渲染检查点	跟踪渲染环节的完成状态
文件上传	向当前项目工作区上传源文件（如剧本、小说文本）
供应商设置	在界面中配置对话、图像、视频、embedding 与 reranker 服务
主题	明暗两套界面主题

7.3 远程访问与安全

Web 服务默认只绑定 `127.0.0.1`，即仅本机可访问；这是有意设计，官方定位其为本地使用或受信任的 SSH 隧道场景。当 ViMax 部署在远程服务器时，在本地电脑建立端口转发：

```
ssh -N -L 4173:127.0.0.1:4173 <user>@<server>
```

`-N` 表示不执行远程命令、仅做端口转发，`-L 4173:127.0.0.1:4173` 表示把远程服务器的 4173 端口映射到本地 4173。建立隧道后，本地浏览器访问 `http://127.0.0.1:4173` 即可。不建议将服务直接暴露到公网：页面中包含密钥配置入口，公网暴露等同于交出配置界面。

7.4 生产构建模式（可选）

开发模式（`npm run dev`）适合日常使用。如需以构建产物方式启动，先构建再使用 `web start` 模式：

```
cd web
npm run build
cd ..
./vimax web start
```

启动脚本会检查 `web/dist/index.html` 是否存在，缺失时报错并提示先执行构建。

本章小结：Web UI 与 TUI 同源同配置，浏览器中完成项目管理与进度预览；默认仅监听本机，远程访问走 SSH 隧道；端口与构建模式各有对应命令。

第 8 章 Novel2Video：长篇小说的分集改编

一部几万字的小说无法直接塞进 Idea2Video——它的输入是一句话的创意，也不是一份完整的剧本。Novel2Video 面对的就是这个规模问题：先压缩叙事、再规划分集，把长文本改造成可逐集拍摄的分集视觉内容。本章讲解该工作流的输入准备、处理逻辑与使用要点。完成本章后，读者应能把一份小说文本转化为分集规划，并理解每集产物的来源。

8.1 工作流定位

Novel2Video 通过三个核心手段处理长文本：

1. **叙事压缩（narrative compression）**：在保留主线与关键情节的前提下压缩篇幅，使小说能进入分镜级规划；
2. **角色追踪（character tracking）**：跨章节维护角色清单与状态，保证第 1 集与第 5 集中的同一角色可以被一致地呈现；
3. **场景规划（scene planning）**：将压缩后的事件流切分为集与场景，衔接分镜与渲染。

8.2 输入准备与流程

小说文本需放置于会话工作区的规范路径：

```
.working_dir/<session_id>/novel2video/novel/novel.txt
```

处理流程的 DAG 前段如下（完整节点序列以仓库 `prompts/workflow.md` 为准）：

```
novel_text（小说原文）
  -> compressed_novel（压缩后的叙事）
  -> events（事件流）
  ->  （后续为角色、分集、分镜等规划环节，继而进入渲染）
```

在 TUI 或 Web UI 中，直接以自然语言下达任务即可，例如「根据这段小说做分集视频规划」，运行时会自行选择 `vimax_novel_planning` 工具；使用者无需记住工具名。

8.3 使用要点

以下要点为基于工作流机制的操作建议：

1. **分集粒度先行确认。**规划阶段产出分集结构后，先审阅分集边界是否符合预期，再进入渲染；分集不合理时修改成本远低于渲染后返工。
2. **篇幅与预算正相关。**事件数量决定镜头总量，镜头数量决定图像与视频 API 调用总量；首次尝试建议选取单章或短篇节选。
3. **文本编码注意。**中文小说文本保存为 UTF-8 编码，避免因编码识别问题导致解析异常。

本章小结：Novel2Video 的三步核心是压缩、追踪、分集；输入有固定路径规范；先确认分集结构再渲染，可把返工成本压到最低。

第 9 章 产物体系与断点续跑机制

随着使用深入，产物目录会越来越复杂：多个会话、多条工作流、每条工作流十来个环节的文件交织在一起。理解产物体系，是把 ViMax 从「能跑通」提升到「可控可复用」的关键，也是断点续跑、局部重生成与故障定位的基础。完成本章后，读者应能画出任意会话的产物地图，并安全地执行中断续跑与局部重生成。

9.1 两级目录：索引与产物

运行时数据分两级存放：

```
.vimax/
  sessions.json          # 会话索引（哪些会话、哪个是当前活跃会话）
  memory.md              # 用户偏好记录
.working_dir/
  <session_id-or-run_id>/   # 会话目录，产物权威所在地
    idea2video/
    script2video/
    novel2video/
```

官方规范明确：`.working_dir/<session_id>/` 是产物的唯一权威位置；`.vimax/sessions.json` 只是索引，`.vimax/memory.md` 只存用户偏好。删除会话时应连同其产物目录一起处理；反之，仅删除索引而保留产物目录会造成索引与产物不一致。

9.2 断点续跑的原理

每个环节开始执行前，先检查自己的产物文件是否已存在；存在则打印跳过日志，直接进入下一环节。这个朴素的「存在即跳过」设计带来三个实际收益：

1. **API 超时、限流后重跑不重复计费**：已付费生成的环节不会重新生成；
2. **手动中断（Ctrl-C）安全**：已完成环节的产物即时落盘，随时可续；
3. **失败定位简单**：按 DAG 顺序找到第一个缺失的产物文件，中断点就在它对应环节。

9.3 局部重生成的操作

两种常见需求与对应操作：

需求	操作	原理
修改上游文本后让下游重生成	手动编辑产物（如修改某场景的分镜）后，删除该场景下游已生成的产物文件（首末帧、镜头视频），再重跑	跳过机制只看文件是否存在，删除即触发重生成
只重渲染、不重规划	保留全部文本产物，删除关键帧与视频类产物后重跑	规划环节产物齐全即被跳过，直接进入渲染

删除产物文件会触发重新生成并再次计费，执行前应确认目标文件确属需要重生成的范围；会话目录在删除前建议整体备份。

9.4 用 view_image 检查画面

TUI 与 Web UI 中可使用 `view_image` 工具查看会话工作区内的图像产物。官方规范要求：对已生成画面的检查、比较、诊断与修订意见，应基于工具返回的实际像素，而不是凭文件名或提示词推断；该工具只读不写，不会修改产物。这一约束同样适用于使用者人工检查——判断某张关键帧是否可用，直接打开图片，不要只看文件名。

本章小结：产物权威在会话目录，索引在 `.vimax/`；跳过机制让中断重跑只补缺、不重复计费；局部重生的本质是「删下游产物、触发重生成」；检查画面要看像素本身。

第三部分 高级篇

第 10 章 视觉一致性调优：角色参考图、首帧与关键帧挑选

跑通流程之后，多数使用者遇到的第一个质量问题几乎都是同一个：主角在第 1 个镜头与第 8 个镜头里判若两人。跨镜头一致性是 ViMax 架构设计的核心命题，本章拆解它的一致性机制，并给出可操作的调优手段。完成本章后，读者应能诊断一致性问题的来源，并选择相应手段改善。

10.1 一致性问题的来源

单张图像由模型随机生成，同一角色描述在不同调用中可能产生完全不同的长相。ViMax 用机制压低这种随机性，机制包括：

1. **角色三视图参考图**：为每个可见角色生成正面、侧面、背面三张参考图并登记注册，后续生成该角色的画面时以参考图为条件；
2. **首帧驱动**：每个镜头先定首帧（first frame，镜头起始画面），再由视频模型从首帧外推动态，画面起点被锚定；
3. **机位树（camera_tree）**：镜头拆解后建立机位连续性结构，相邻镜头的视角与空间关系有据可依；
4. **依赖感知的状态追踪**：按论文描述，框架跨时间边界追踪角色与环境状态，并由 VLM 持续监测修正。

10.2 关键帧挑选：num_candidates

关键帧质量直接决定镜头质量。ViMax 支持为关键帧并行生成多张候选，再由具备视觉能力的对话模型充当裁判挑选最优者。配置位于 Agent 模式的 `agent.local.yaml`：

```yaml
image_selection:
  num_candidates: 2 # 设为 1 可关闭挑选
```

`num_candidates` 的含义与代价：

取值	行为	代价
2（默认）	每个关键帧生成 2 张候选并挑选	图像调用次数翻倍，费用相应增加
1	关闭挑选，直接采用唯一生成结果	费用减半，质量下限风险升高

10.3 调优手段清单

以下手段按生效位置排列，均为操作建议：

1. **在源头细化角色描述。**创意或剧本中给角色留下具体、稳定的外形描述（发色、服饰、体型），三视图与参考提示词会继承这些描述；「一个女孩」远不如「齐肩黑发、红色连帽衫的瘦高女孩」稳定。
2. **减少跨镜头的角色与场景切换。**场景与镜头数量越多，一致性风险点越多；在 `user_requirement` 或对话中约束规模，是最直接的风险控制。
3. **开启或保持关键帧挑选。**预算允许时保持 `num_candidates: 2`，让视觉裁判过滤明显跑偏的首帧。
4. **先审后渲。**利用 Agent 模式的分阶段特性，文本规划完成后先人工审阅分镜与镜头描述，再进入渲染；画面问题往往在规划阶段已可预见。
5. **局部重生成替代全量重跑。**个别镜头跑偏时，按第 9 章方法删除该镜头的产物定点重生成，而非整体重跑。

本章小结：一致性靠参考图、首帧与机位连续性三层机制保障；`num_candidates` 用费用换质量下限；描述具体化、规模受控、先审后渲、定点重生成，是四条最有效的调优路径。

第 11 章 模型选型与成本控制

ViMax 的费用不发生在软件本身，而发生在它调用的每一次 API 上：规划要调对话模型，每个参考图与关键帧要调图像模型，每个镜头要调视频模型。不了解成本构成就按默认参数跑长片，账单可能远超预期。本章给出成本构成模型、选型原则与一套可执行的控制手段。完成本章后，读者应能在开跑之前估算一次生成的账单构成，并主动控制其规模。

11.1 成本构成

一次 Idea2Video 运行的费用由三部分叠加：

环节	调用对象	次数量级
叙事规划（简报、角色、剧本、分镜、镜头拆解、机位、提示词）	对话模型	数十次文本调用
角色参考图	图像模型	每个可见角色 3 张（三视图）
关键帧	图像模型	每镜头至少 1 张；`num_candidates: 2` 时为 2 张
镜头视频	视频模型	每镜头 1 段
挑选裁判（可选）	对话模型（视觉）	每关键帧 1 次

镜头数是总费用的乘数：镜头数翻倍，关键帧与视频调用近似翻倍。这就是各章反复强调约束场景数与镜头数的原因。具体单价因供应商、模型与时段而异，且随市场变化，本教程不列具体数字，开跑前应在所选供应商控制台查询当前价格。

11.2 选型原则

结合第 3 章的建议，从成本视角再总结一次：

1. **对话模型从低**：规划与裁判任务对模型能力的峰值要求不高，Flash 级轻量模型即可胜任，把预算省给生成环节；
2. **图像与视频从高**：这两类直接决定成片观感，且调用次数可预估，值得集中预算；
3. **优先走聚合网关试错**：OpenRouter 一类网关允许用一份密钥切换多种模型，试错期切换成本低；确定主力组合后再考虑直连。

官方明确支持的组合见第 5.3 节表格；MiniMax H3 视频生成与可组合 Skills 在路线图中尚未标记完成，选型时不应将其计入现有能力。

11.3 控制手段清单

手段	操作	生效位置
硬约束规模	`user_requirement` 写明场景与镜头上限	脚本直跑
小规模起步	对话中不主动要求扩量，接受默认的 1 场景 3–5 镜头	Agent TUI / Web UI
关闭挑选	`image_selection.num_candidates: 1`	Agent 模式配置
复用产物	中断续跑、局部重生成，避免全量重跑	全部入口
先文本后渲染	规划完成后人工审阅再渲染，避免成片后返工	Agent 模式
分层试跑	先用最小配置验证全流程，再升级模型与规模	全部入口

本章小结：费用等于三类模型调用之和，镜头数是最大乘数；对话模型走低、图像视频走高；控制规模、复用产物、先审后渲，是三条不牺牲成片质量的前提下最有效的省钱路径。

第 12 章 故障排查与常见问题

ViMax 环节多、依赖外部服务，出错是常态而非例外。好消息是：每步落盘的产物体系让故障定位有迹可循。本章整理高频问题的现象、原因与处理方法，并给出通用的排查路径。完成本章后，读者应能独立处理安装、配置、运行与质量四类常见故障。

12.1 通用排查路径

按以下顺序定位问题，多数故障可在三步内定位：

1. **确定中断环节**：按 DAG 顺序检查产物目录，第一个缺失的产物文件对应中断环节；
2. **阅读终端输出**：跳过日志、报错原文通常直接指明原因（密钥无效、依赖缺失、端口占用等）；
3. **核对配置**：确认 `agent.local.yaml` 或对应 yaml 的密钥、`base_url` 与 `model_provider` 相互匹配。

12.2 常见问题对照表

现象	原因	处理
`vimax: command not found`	未在仓库根目录，或非 bash 环境	在仓库根目录执行 `./vimax …`；Windows 在 Git Bash 或 WSL 中执行
报错 `ui dependencies are missing`	TUI 前端依赖未安装	执行 `cd ui && npm install`
报错 `web dependencies are missing`	Web 前端依赖未安装	执行 `cd web && npm install`
报错 `web build is missing`	未构建即使用 `vimax web start`	先 `cd web && npm run build`
API 401、鉴权失败	密钥错误或未生效	检查密钥；确认环境变量名正确（`VIMAX_LLM_API_KEY` 等)
生成中途超时或限流	供应商限流或网络波动	直接重跑，跳过机制自动续；必要时降低镜头数
浏览器无法打开 Web UI	服务只监听 127.0.0.1，或端口被占用	本机访问确认地址；远程走 SSH 隧道；换 `VIMAX_WEB_PORT`
跨镜头角色不一致	一致性机制外的随机因素	按第 10 章手段处理
找不到 AutoCameo 入口	该能力尚无公开实现	见 5.4 节，勿按特性列表默认可用
产物目录混乱、找不到文件	手动在根级建立 `.working_dir/idea2video/` 等路径	遵守会话目录规范（第 6.4、9.1 节）

12.3 求助渠道

自查无果时，可到仓库 Issues 页面（https://github.com/HKUDS/ViMax/issues ）检索或提交问题。提交前建议附上：ViMax 版本（`git log -1` 或 Release 版本号）、操作系统、报错原文、中断环节对应的产物目录状态（注意隐去密钥）。仓库迭代较快，提交前先确认问题在最新 main 分支上仍可复现。

本章小结：故障定位的通用路径是「按 DAG 找第一个缺失产物、读报错、核配置」；高频问题集中在依赖安装、密钥配置与端口监听三点；提问前先在最新代码上复现。

第 13 章 综合实战：从创意到成片的完整项目

前面的章节分别覆盖了安装、配置、三条工作流与调优排障，本章把它们串成一个完整项目：在 Agent TUI 中，从一个中文创意出发，经过讨论、规划、审阅、渲染与局部迭代，交付一部短片。完成本章后，读者应能把全书知识组织成一套可重复的个人创作流程。

13.1 项目设定

示例项目：一部面向短视频平台的卡通风格小短片。

- 创意：「一只总在深夜便利店铺货的流浪猫，与夜班店员之间形成了默契，直到某晚猫没有出现。」
- 工作流选择：只有创意、没有剧本，选择 Idea2Video；
- 规模约束：面向移动端观看，控制在 1 至 2 个场景、10 个镜头以内；
- 风格：`Cartoon`。

13.2 流程走查

第一步，环境与配置检查。确认 `uv sync` 与 `ui` 依赖就绪，`configs/agent.local.yaml` 三段配置可用，密钥走环境变量。

第二步，进入 TUI 并新建会话。

```
./vimax tui new
```

第三步，提交创意并确认工作流。

```
使用者：创意如下：一只总在深夜便利店铺货的流浪猫，与夜班店员之间形成了默契，
        直到某晚猫没有出现。请控制在 1 到 2 个场景、10 个镜头以内，卡通风格。
Agent：请确认使用哪条工作流：idea2video、script2video 还是 novel2video?
使用者：idea2video。
```

第四步，审阅文本规划。规划完成后，逐一检查 project_brief、characters、script、storyboard：角色描述是否具体稳定、场景切分是否合理、镜头是否在约束数量内。发现问题时在此阶段修订，成本最低。

第五步，确认渲染。审阅通过后明确指示开始渲染，等待关键帧与镜头视频逐镜头生成。

第六步，成片检查与局部迭代。观看 `final_video.mp4`；若第 5 个镜头跑偏（例如猫的花色变化），用 `view_image` 核对该镜头关键帧，删除该镜头的首末帧与视频产物后重跑，实现定点重生成（见第 9.3 节）。

第七步，归档。会话可保留以便日后恢复；成片与产物位于 `.working_dir/<session_id>/idea2video/`，重要成片另行备份到项目目录之外。

13.3 项目复盘清单

每个项目收尾时过一遍以下清单，沉淀为个人流程：

1. 创意是否提供了具体冲突，而非单一画面设想；
2. `user_requirement` 或对话约束是否写明了场景与镜头上限；
3. 文本规划是否经过人工审阅后才进入渲染；
4. 一致性问题的镜头是否采用局部重生成而非全量重跑；
5. 本次的实际调用构成与预估是否一致，下一步选型是否需要调整。

本章小结：完整项目的七个步骤是「检查—建会话—提交并确认—审规划—渲染—迭代—归档」；复盘清单把单次经验转为可重复流程。

附录

附录 A 配置速查表

Agent 模式配置（configs/agent.local.yaml）

字段	说明
`llm.model_provider`	供应商协议，`openai` 表示 OpenAI 兼容协议
`llm.model` / `image.model` / `video.model`	各类模型标识
`llm.base_url` / `image.base_url` / `video.base_url`	各类服务地址
`llm.api_key` / `image.api_key` / `video.api_key`	各类服务密钥（也可用环境变量）
`image_selection.num_candidates`	关键帧候选数，默认 2，设 1 关闭挑选

脚本直跑配置（configs/idea2video.yaml 等）

字段	说明
`chat_model.init_args.*`	对话模型的 model、model_provider、api_key、base_url
`image_generator.class_path` / `init_args`	图像适配器及其参数
`video_generator.class_path` / `init_args`	视频适配器及其参数
`working_dir`	产物落盘目录

环境变量

变量	作用
`VIMAX_LLM_API_KEY`	对话模型密钥
`VIMAX_IMAGE_API_KEY`	图像生成密钥
`VIMAX_VIDEO_API_KEY`	视频生成密钥
`VIMAX_WEB_PORT`	Web UI 端口（默认 4173）

常用命令

命令	作用
`uv sync`	创建虚拟环境并安装依赖
`uv run python main_idea2video.py`	脚本直跑 Idea2Video
`uv run python main_script2video.py`	脚本直跑 Script2Video
`./vimax tui [new/resume <id>]`	启动 TUI 及会话管理
`./vimax web`	启动 Web 开发服务器
`./vimax web start`	以构建产物启动 Web（需先 `npm run build`）
`ssh -N -L 4173:127.0.0.1:4173 <user>@<server>`	远程端口转发

附录 B 术语表

术语	英文	定义
智能体	Agent	能理解意图、规划步骤并调用工具完成任务的程序
智能体循环	Agent Loop	使用者与智能体多轮对话、逐步推进任务的交互模式
终端用户界面	TUI	在终端中运行的交互界面
有向无环图	DAG	节点单向连接、不构成回路的结构，用于描述流水线环节依赖
工作流	Workflow	从特定输入到成片的完整环节序列，如 Idea2Video
产物	Artifact	流水线各环节落盘生成的文件，是续跑与重生成的依据
会话	Session	一次交互创作的容器，含对话记录与产物目录
项目简报	project_brief	Idea2Video 首个环节产出的项目概述
分镜	storyboard	将故事转化为逐画面视觉叙事的设计稿
镜头拆解	shot_decomposition	将场景细分为具体镜头的环节
机位树	camera_tree	描述镜头间机位与视角连续性的结构
首帧	first frame	镜头起始画面，作为视频生成的锚点图像
关键帧	keyframe	决定镜头画面基础的关键图像，由图像模型生成
三视图	three-view reference	角色正面、侧面、背面三张参考图，用于外观一致性
适配器	adapter	对接具体生成服务的封装类，以 class_path 指定
视觉语言模型	VLM	同时理解图像与文本的模型，此处用于画面检查与挑选
检索增强生成	RAG	生成时检索相关背景材料以提高连贯性的技术
确认门槛	confirmation gate	要求使用者显式确认工作流后才能调用规划工具的机制
断点续跑	resumable run	依据已存在产物跳过完成环节、只补缺失环节的机制
叙事压缩	narrative compression	在保留主线前提下压缩长文本篇幅的处理
端口转发	port forwarding	将远程端口映射到本地的网络手段，此处指 SSH 隧道